

Parallel Lenia using OpenMPI

Jakob Drusany, Yon Ploj

Abstract—We implement parallel Lenia simulations using OpenMPI with strip decomposition. We benchmark the implementation on grid sizes from 256×256 to 4096×4096 and compare in terms of speedup relative to sequential execution. We found significant (over 3000%) speedup that increases monotonously with the number of processing units allocated.

I. INTRODUCTION

Lenia is a continuous cellular automaton inspired by Conway’s Game of Life. Lenia evolves according to continuous-valued convolution-based update rules and therefore requires significantly more computation.

Because every simulation step requires a convolution over the entire grid, the computational complexity grows quadratically with grid size. The embarrassingly parallel nature of the convolution operation over a large grid makes Lenia a suitable candidate for distributed-memory parallelization using MPI.

In this work, we implement a parallel Lenia simulator using OpenMPI and evaluate it on grids up to 4096×4096 .

II. IMPLEMENTATION DETAILS

We divide the work into horizontal strips. Neighbouring strips share an overlap equal in height to half of the kernel size.

$$\text{overlap} = \left\lceil \frac{\text{kernel_height}}{2} \right\rceil$$

This allows us to perform the convolution (and later also the Lenia-evolution) locally within each process, then only sync the overlap regions once per iteration.

- 1) The process places orbiums that fall into its own region according to the configuration.
- 2) Overlap regions are exchanged with neighbouring ranks above and below using `MPI_Isend` and `MPI_Irecv`.
- 3) The local world is convolved and Lenia-evolved.
- 4) If assembling a GIF, the results (local world) of each process are gathered in process 0 and the frame is saved.
- 5) Repeat from step 2 for as many iterations as desired.

III. EXPERIMENTAL SETUP

A. Grid configurations

We tested our algorithm on grid sizes: 256×256 , 512×512 , 1024×1024 , 2048×2048 and 4096×4096 . Each test runs 100 simulation steps with $dt = 0.1$ and kernel size 26×26 . Measurements include all relevant data transfers and are averaged over 5 runs.

B. Upper process count limit

If the width of one section ($\frac{N}{\text{core_count}}$) falls below half of kernel height, the overlap would intersect more than one section. Because of this, we imposed an upper core count limit for grid size N of $\left\lfloor \frac{2 \cdot N}{\text{kernel_height}} \right\rfloor$ in order to assure we only need to send the overlap data to one neighbour above and below. This makes 16 and 32 cores invalid on a 128×128 grid size.

C. Benchmark Methodology

Execution time was measured from the beginning of the simulation loop until completion of the final iteration. Initialization overhead and GIF export times were not measured.

IV. RESULTS

A. Execution Time

In table I we observe the speedup produced by dividing the image into strips and dividing them among multiple processes. The effect is noticeable regardless of grid size.

For the smallest grid (128×128), the division into 16 and 32 strips is inapplicable due to the fact that the width of a single strip becomes thinner than the size of the overlap region.

In the sequential measurement of the largest grid (4096×4096) the algorithm time limit was exceeded (denoted TLE in the table).

Table I
MEAN MPI EXECUTION TIME (IN SECONDS) ACROSS GRID DIMENSIONS

#proc	128	512	1024	2048	4096
1	4.678	74.855	299.358	1197.159	TLE
2	2.343	37.450	149.781	598.821	2397.666
4	1.184	18.746	74.956	299.610	1197.918
16	–	4.776	18.815	75.086	300.065
32	–	2.424	9.443	37.758	150.152

To provide a meaningful benchmark despite the exceeded time limit, we inferred the expected runtime by fitting a parabola to the rest of the measurements (Fig. 1). The estimated running time was approximately 1 hour and 23 minutes.

Table II
MEAN MPI SPEEDUP ACROSS GRID DIMENSIONS

#proc	128	512	1024	2048	4096
2	209.2%	207.6%	207.4%	207.5%	207.3%
4	414.0%	414.8%	414.5%	414.8%	415.0%
16	–	1628.1%	1651.4%	1655.2%	1656.8%
32	–	3207.6%	3290.5%	3291.5%	3310.9%

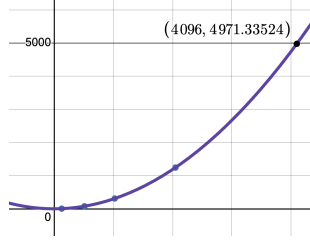


Figure 1. Sequential algorithm processing time (in seconds) for different grid sizes

V. CONCLUSION

We implemented parallel Lenia simulations using OpenMPI. Our implementation shows significant improvement compared to sequential execution.

Further work could focus on increasing the size of the overlap regions to reduce the number of data exchanges between processes and thus reduce synchronization overhead.

It would also be worth exploring different methods of division of labour among processes to optimize the amount of work allotted per process.